言語処理系 第9回
関数呼び出し規約

木村啓二
keiji@waseda.jp

第7回演習問題(1)

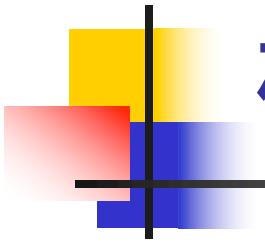
- 先のwhile文の変換に倣ってdo-while文の四組列を書け
  - do S while (E);が与えられるものとする。
- 同じくdo-while文のコード生成関数gen_dowhile()を書け

```
(label1:)
S
(jf E label2)
(jmp label1)
(label2:)
```

```
gen_dowhile()
{
  l_begin = get_label();
  l_exit = get_label();
  gen_label(l_begin);
  Sの生成
 条件式Eの生成
  l_exitを使ってjf文を生成
  l_beginを使ってjmp文を生成
  gen_label(l_exit)
}
```

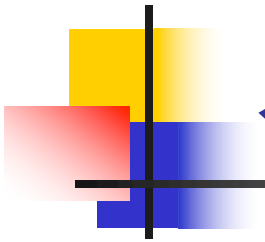
（コード生成の続き）配列要素のアドレス計算

- intの配列aの要素a[10]のアドレス計算は？
  - &a[0]+10*sizeof(int)
  - Cの文法ではsizeof(int)はつけないが、ここでは要素のサイズを意識してほしいのであえてつけている
- int b[5][10]の場合のb[1][3]のアドレス計算は？
  - &b[0][0]+1*10*sizeof(int)+3*sizeof(int)
- 一般形int a[n1][n2]…[nk]のa[i1][i2]…[ik]は？
  - &a[0][0]…[0]+i1*n2*…*nk*sizeof(int)+i2*n3*…*nk*sizeof(int)+…+ik*sizeof(int)
  - もちろん、定数項はあらかじめ計算してしまう
- 以上の式を、前回のように中間表現に変換する
- それではint **pにおけるp[1][3]は？
  - p1 = p+sizeof(int*)*1; p2 = *p1+sizeof(int)*3;



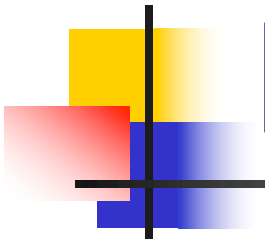
構造体のアドレス計算

- struct sample { int a; char b; int c } st1;のst1.cのアドレス計算は？
  - &st1+cの基底アドレスからのオフセット
- struct sampleの各メンバa, b, cのオフセットを決定する必要がある
- intが4バイト、charが1バイトだとすると、それぞれ+0, +4, +5?
  - 多くのCPUではこのようにならない
  - 整列（alignment）のルールにより、+0, +4, +8となることが多い



メモリの整列

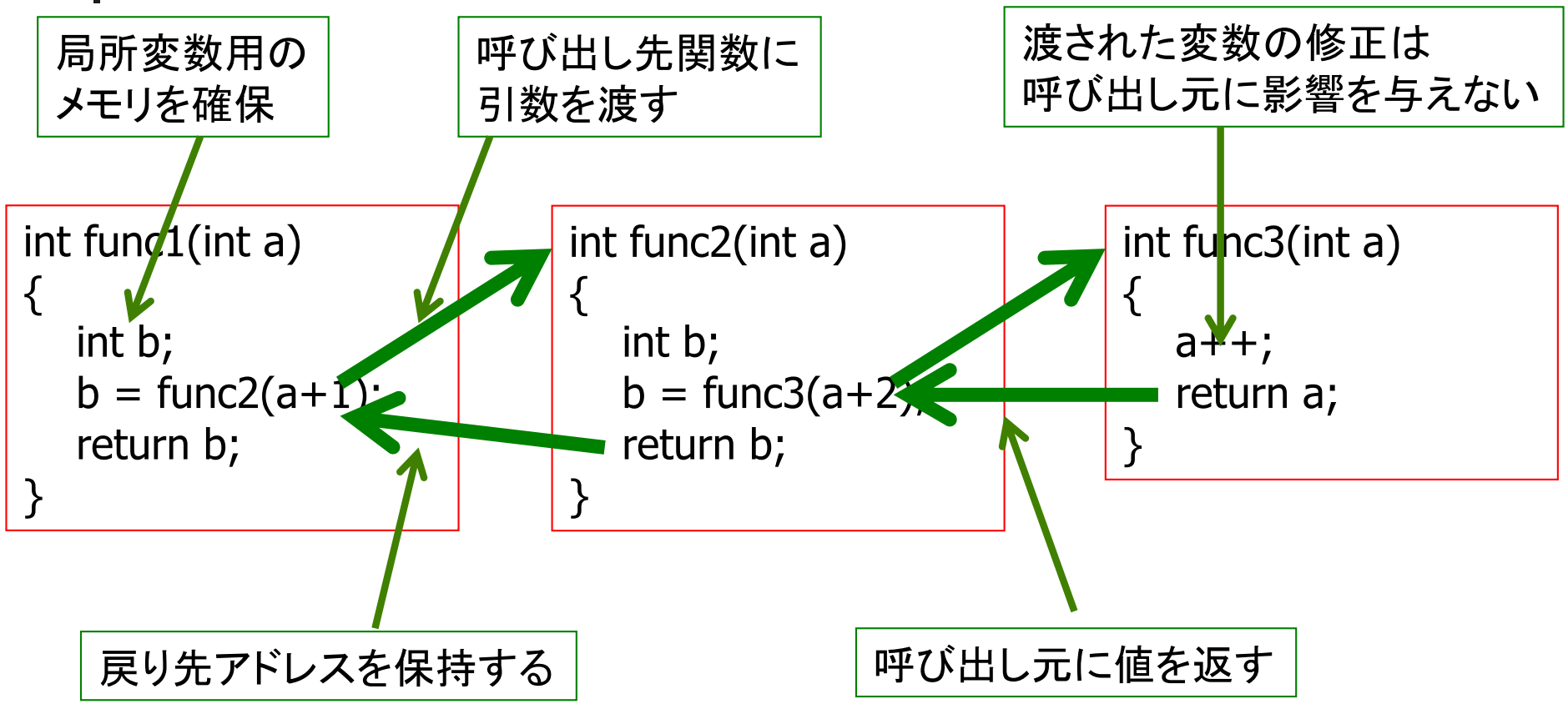
- ハードウェアの都合により、データが配置可能なアドレスには制約がある
  - int等の4バイトデータは4の倍数のアドレスに配置する必要がある
  - double等の8バイトデータは8の倍数のアドレスに配置する必要があるか、4の倍数に配置しても良いが8の倍数の方が性能的に良い
    - CPUの種類によって異なる
  - 先の例では、メンバcは4の倍数の位置に配置されなければならない
    - bとcの間に隙間（pad）が生じる
- 整列を考慮した変数の配置もコンパイラの役割



関数呼び出し

- スタックフレームに引数などをセットしていく
- スタックフレームの具体的なレイアウトは CPUやOSごとに決められている
  - application binary interface (ABI)
- 引数はなるべくレジスタを利用して授受される
  - ABIで使用されるレジスタが規定される

関数(function, procedure)呼び出しの概要





関数呼び出しのステップ

- スタックを使って局所変数、引数、戻りアドレスを管理する
  - 関数呼び出し毎にスタックが拡張され、その関数用メモリ領域（スタックフレーム）を確保する
- スタックフレームに引数を書き込む
  - いくつかの引数はレジスタを使って渡されることもある
- 関数を呼び出す（ジャンプする）
- 呼び出し先の関数用メモリ領域を確保する
- 関数本体を実行する
- 返値を呼び出し元がアクセスできる場所に書き込む
  - Cの場合、通常はレジスタ
- 呼び出し元に返る（ジャンプする）

参考：利用するARM64命令
関数呼び出し関連命令

- stp x29,x30, [sp, <定数>]!
  - sp+<定数>を新たなspの値とする
  - 更新されたspの指すアドレスにx29とx30を格納する
  - スタックにx29,x30をpushするのと同じ
- ldp x29,x30,[sp],<定数>
  - spの指すアドレスからx29とx30を読み出し、spの値をsp+<定数>に更新する
  - スタックからx29,x30をpopするのと同じ
- bl <ラベル>
  - bl命令の次の命令のアドレス（PC+4）をx30に格納し、<ラベル>にジャンプする
- ret
  - x30に格納されているアドレスにジャンプ（リターン）する（関数から戻る）

ARM64における関数の呼び出し

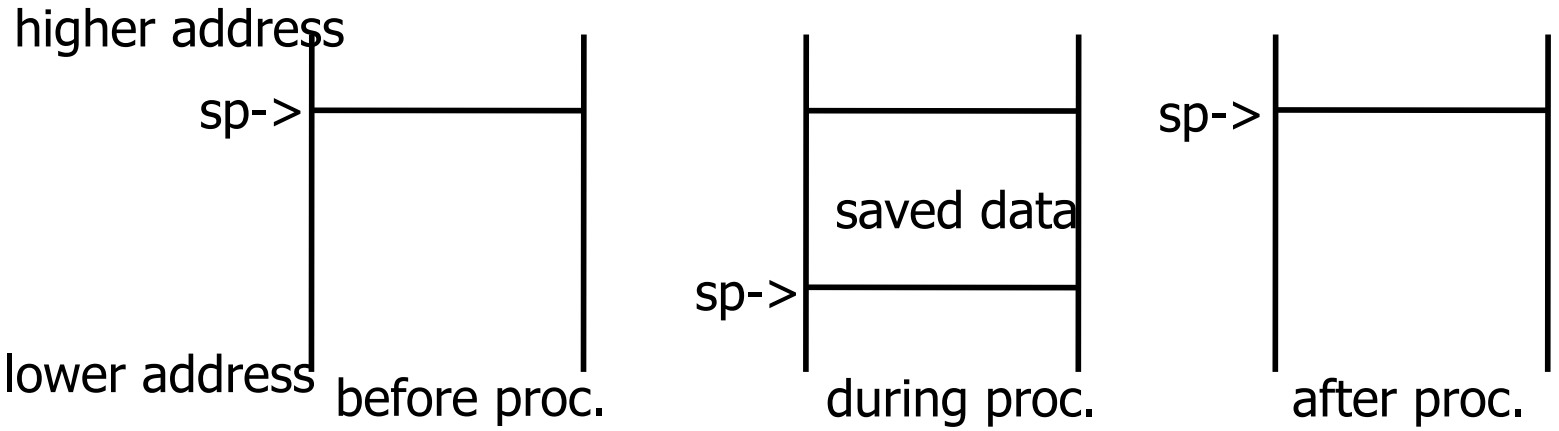
- 引数(argument)をレジスタ及びスタックに保存する
  - 8個までレジスタ経由で渡す
    - スタックへの保存は呼び出された関数で必要に応じて行う
  - 9個目以降の引数はスタック経由で渡す
- 関数の呼び出し：bl <ProcedureAddress>
  - bl命令の次の命令のアドレスをx30に保存し、ProcedureAddressにジャンプする
- 関数用メモリ領域確保
  - stpでx29,x30を保存し、spの値を確保すべき領域分ずらす
- 結果をレジスタw0にセットする
- 関数用メモリ領域解放
  - spの値を戻し、ldpでx29,x30も戻す
- 呼び出し元に戻る：ret
  - x30に格納されたアドレスにより、呼び出しもとに戻る

ARM64のstp,ldp命令と関連する特殊なアドレッシング

- プレインデックス
  - …[reg,offset]!
  - regをreg+offsetの値に更新してからメモリアクセス
- ポストインデックス
  - …[reg],offset
  - メモリアクセスをしてからregをreg+offsetの値に更新
- stp x29,x30,[sp,<定数>]!
  - spをsp+<定数>に更新してから、spの位置にx29,x30をストア
- ldp x29,x30,[sp],<定数>
  - spの位置からx29,x30をロードしてから、spの値をsp+<定数>に更新

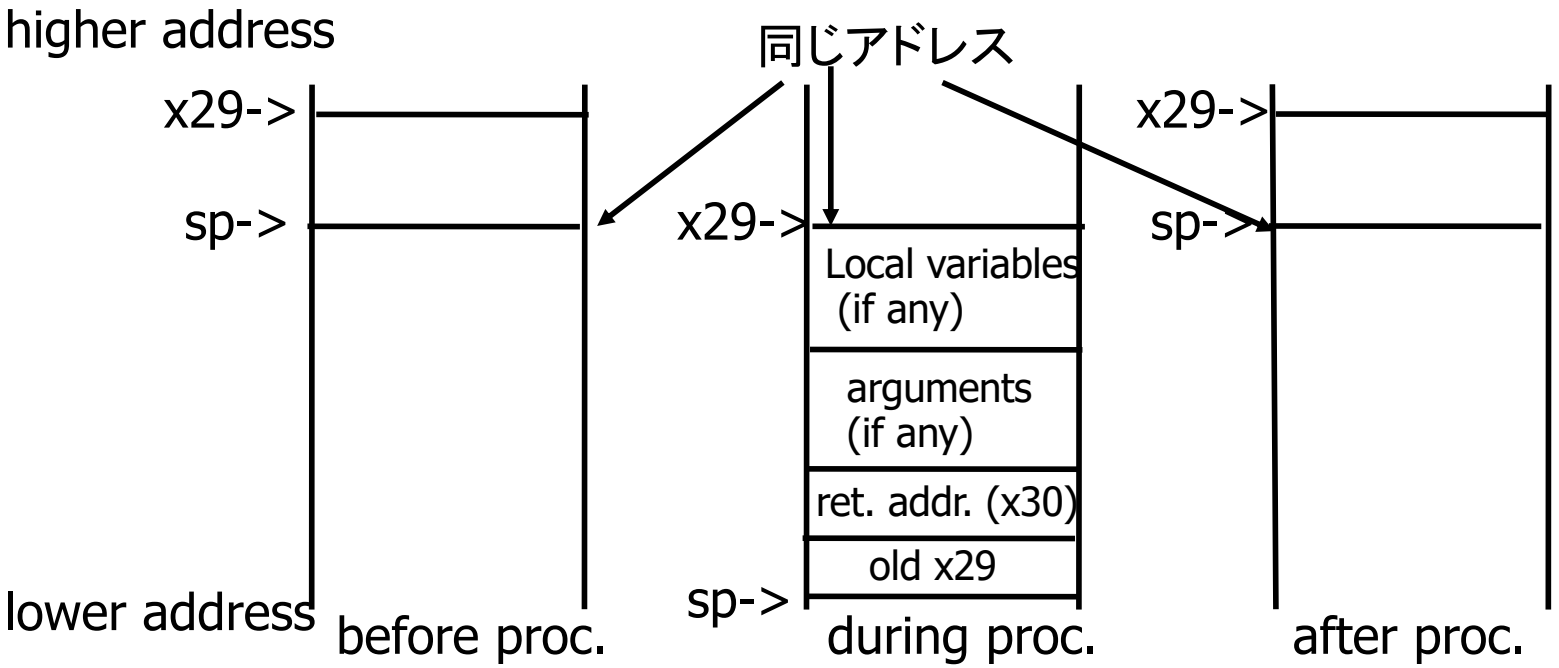
スタック（復習）

- プッシュ（push）：スタックに値を保存する
- ポップ（pop）：スタックから値を取り出す
- スタックポインタ（stack pointer）：スタックの先頭（頂点）アドレスを保持するポインタ



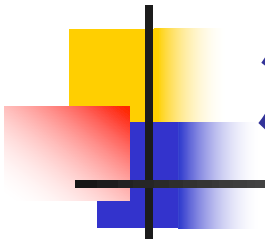
スタックフレーム (活性レコード: Activation record)

- 局所変数等、関数実行に必要な情報用の領域
- x29 (ARM64でのフレームポインタ: frame pointer、他の命令セットではfpと表される場合が多い): フレームの底を指すポインタ
  - spとの違いに注意
  - tlcのレイアウトに関しては、arch_arm64.cのコメントを参照



push/popを用いた関数呼び出しの例

- func(int a, int b)
  {
  int c;
  c = a+b;
  return c;
  }
- func:
  stp x29,x30,[sp,-32]! # 古いx29とx30（戻りアドレス）のスタックへの保存（push）
  # sp<-sp-32
  add x29,sp,32 # x29を古いspの位置に合わせる
  str w0,[x29,-8] # 引数aの保存
  str w1,[x29,-12] # 引数bの保存
  ldr w8,[x29,-8] # 引数aのロード
  ldr w9,[x29,-12] # 引数bのロード
  add w8,w8,w9 # 関数本体
  str w8,[x29,-4] # 加算の結果を変数cの位置に格納
  ldr w0,[x29,-4] # 戻り値用レジスタにcの値をロード
  ldp x29,x30,[sp],32 # スタックから古いx29,x30を取り出し（popし）、
  #spの値を元に戻す（sp<-sp+32)
  ret # 呼び出し元に戻る



演習問題(1)

- 以下のプログラムを実行した結果表示される値を答えよ。また、その根拠も簡潔に述べよ。

```
int func(int a)
{
    a++;  /* この文の影響がmainにあるかどうか、という問題 */
    return a;
}

int main()
{
    int t, v = 1;
    t = func(v);
    printf("%d¥n", v+t);
    return 0;
}
```

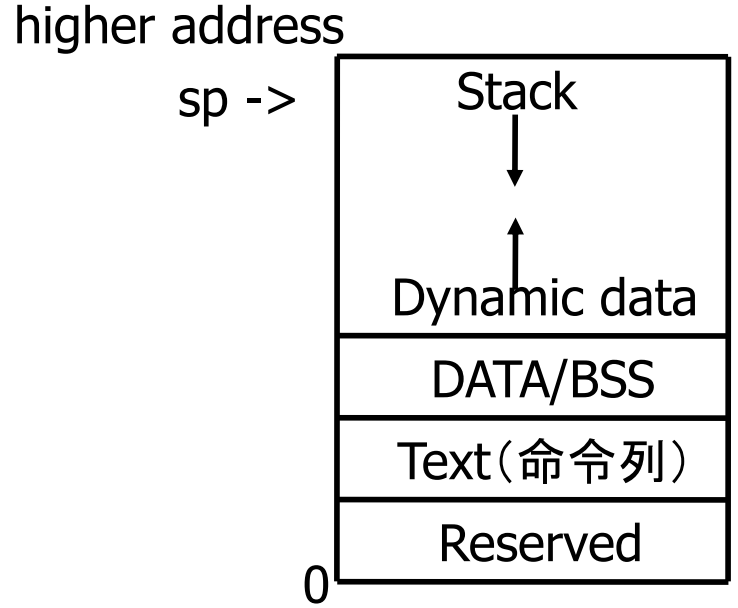
関数呼び出しの入れ子

- 関数から関数を呼び出す場合（再帰呼び出しなど）
  - x29, spその他のレジスタなど、関数実行に必要な情報（コンテキスト：context）はスタックに保存する
- int fact(int n)
  {
  if (n < 1) return 1;
  else return (n*fact(n-1));
  }
- fact:

```
    stp     x29, x30, [sp, -32]! #x29とx30（戻りアドレス）をスタックへ保存
    mov     x29, sp
    str     w0, [sp, 28]   # 引数nの保存
    ldr     w0, [sp, 28]
    cmp     w0, 0   # n > 1 ?
    bgt     .L2
    mov     w0, 1   #return 1
    b       .L3
.L2:
    ldr     w0, [sp, 28]
    sub     w0, w0, #1   #n-1
    bl      fact
    mov     w1, w0
    ldr     w0, [sp, 28] # nの取り出し
    mul     w0, w1, w0   # return n*fact(n-1)
.L3:
    ldp     x29, x30, [sp], 32   #x29とx30をスタックから取り出す
    ret
```

広域変数やヒープ

- Cにおける広域（グローバル）変数や静的（スタティック）変数
  - DATAやBSSと呼ばれる特定の領域に確保される
  - 未初期化変数はプログラム実行時に0で初期化される
- 動的に確保されたオブジェクトはヒープ(heap)を利用する
  - Cのmalloc()やJava, C++のnew



静的変数とグローバル変数

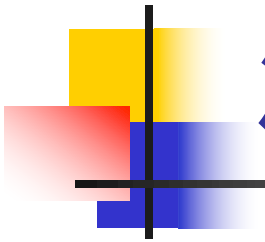
```
#include  <stdio.h>
int gcounter;

int c1()
{
    static int count;
    gcounter++;
    return count++;
}

int c2()
{
    static int count;
    gcounter++;
    return count++;
}

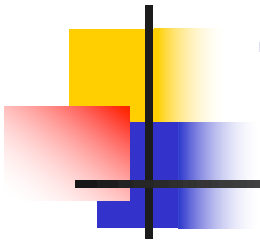
int main()
{
    int i, s1, s2;
    for (i = 0; i < 5; i++) s1 = c1();
    for (i = 0; i < 10; i++) s2 = c2();
    printf("s1 %d, s2 %d, g %d¥n",
            s1, s2, gcounter);
    return 0;
}
```

```
        .comm   gcounter,4,4
c1:
…
        adrp    x0, :got:gcounter
        ldr     x0, [x0, #:got_lo12:gcounter]
        ldr     w1, [x0]
        add     w1, w1, 1
        str     w1, [x0]
        adrp    x0, count.3809
        add     x0, x0, :lo12:count.3809
        ldr     w1, [x0]
        add     w2, w1, 1
        str     w2, [x0]
…
c2:
…
        adrp    x0, :got:gcounter
        ldr     x0, [x0, #:got_lo12:gcounter]
        ldr     w1, [x0]
        add     w1, w1, 1
        str     w1, [x0]
        adrp    x0, count.3812
        add     x0, x0, :lo12:count.3812
        ldr     w1, [x0]
        add     w2, w1, 1
        str     w2, [x0]
…
main:
…
        .local  count.3809
        .comm   count.3809,4,4
        .local  count.3812
        .comm   count.3812,4,4
```



演習問題 (2)

- 先のスライドの実行結果を示せ。また、c1()及びc2()それぞれのcountの値とgcounterの値がなぜそのようになったか、理由を簡潔に説明せよ。



まとめ

- 関数呼び出しとメモリ領域